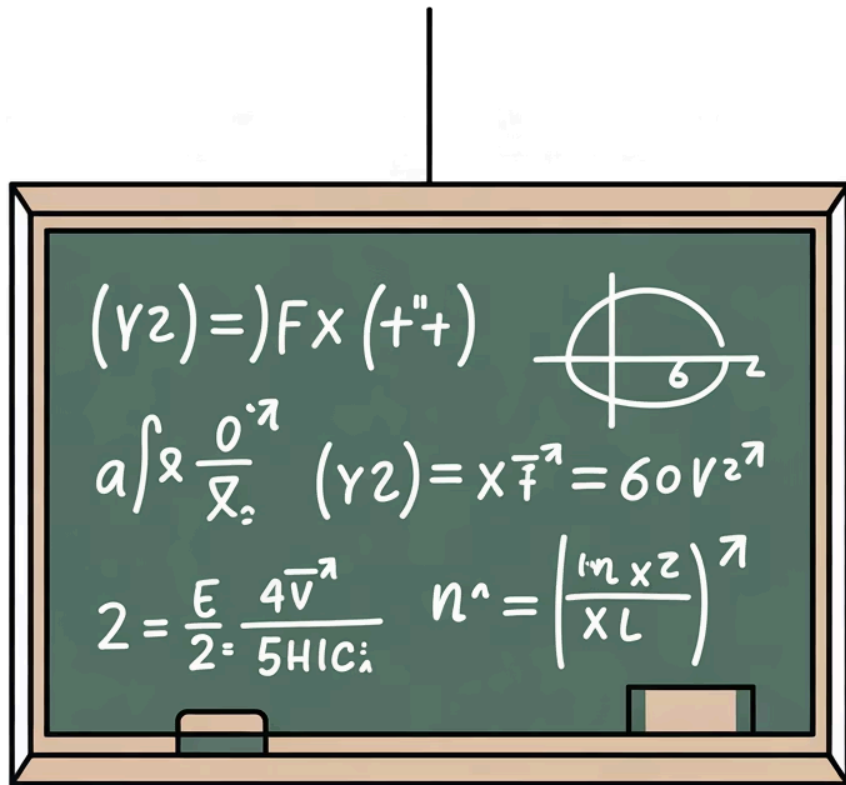




Escrita de Formalização

A comunicação precisa de conceitos matemáticos e computacionais por meio de linguagem formal rigorosa e estruturada.

Eduardo Ogasawara

eduardo.ogasawara@cefet-rj.br
<https://eic.cefet-rj.br/~eogasawara>

O que é Formalização?

A formalização é o processo fundamental de expressar conceitos matemáticos e computacionais usando linguagem precisa, notação padronizada e estrutura lógica rigorosa. Este processo permite a comunicação inequívoca de ideias complexas e a verificação sistemática de resultados.

Na matemática, a formalização transforma intuições em definições rigorosas e argumentos em provas verificáveis. Na ciência da computação, ela permite especificar algoritmos e sistemas com precisão absoluta.

A linguagem formal elimina ambiguidades presentes na linguagem natural, estabelecendo um vocabulário técnico compartilhado pela comunidade científica. Cada símbolo, cada operador e cada construção possui significado bem definido.

Dominar a escrita formal é essencial para estudantes e pesquisadores, pois representa a base para a comunicação científica, o desenvolvimento de teorias e a implementação de sistemas computacionais confiáveis.

Por que Formalizar?

A formalização matemática traz solidez a conceitos abstratos e permite uma comunicação precisa e rigorosa entre pesquisadores e profissionais.

Principais benefícios:

- Ajuda a evitar ambiguidades e imprecisões na comunicação
- Facilita a análise sistemática de problemas e algoritmos
- Permite validar resultados com maior confiança
- Garante uma base sólida para aplicações computacionais

Precisão Operacional

Define com exatidão as operações realizadas

Análise Rigorosa

Permite análise detalhada da complexidade

Clareza Conceitual

Evita ambiguidades e facilita compreensão

Princípios de Escrita Clara

Clareza de Propósito

Defina objetivos explícitos no início, estabelecendo o que será demonstrado e por que é importante.

Estrutura Lógica

Organize o conteúdo em seções coerentes, progredindo das definições básicas aos resultados principais.

Precisão Técnica

Use terminologia rigorosa e notação padronizada, evitando ambiguidades e imprecisões.

Compreensibilidade

Balance rigor com explicações intuitivas, auxiliando o leitor na compreensão dos conceitos.

A escrita eficaz de formalizações matemáticas e computacionais exige domínio tanto do conteúdo técnico quanto das técnicas de comunicação. Textos bem escritos apresentam argumentos de forma linear e progressiva, introduzindo cada conceito no momento apropriado e justificando cada passo com clareza.

É fundamental estabelecer convenções notacionais logo no início do texto e mantê-las consistentemente. Definições devem preceder seu uso, e teoremas devem ser enunciados antes de suas provas. A divisão em seções temáticas facilita a navegação e compreensão, permitindo que leitores identifiquem rapidamente os elementos de interesse.

📄 **Referência:** [2] J. Zobel, 2015, *Writing for Computer Science*. Springer.

Elementos de uma Boa Proposição

A escrita eficaz de proposições requer atenção cuidadosa a múltiplos elementos que garantem clareza, precisão e compreensibilidade. Cada componente desempenha papel crucial na comunicação matemática:

Enunciado Claro e Preciso

A proposição deve ser formulado de maneira inequívoca, estabelecendo claramente as hipóteses e a conclusão. A linguagem deve ser formal, mas acessível ao público-alvo.

Hipóteses Bem Definidas

Todas as condições necessárias devem ser explicitadas completamente, sem deixar pressupostos implícitos que possam gerar confusão ou interpretações errôneas.

Notação Consistente

Utilizar símbolos padronizados e manter consistência na notação ao longo de todo o texto, definindo claramente cada variável e operador introduzido.

Estrutura Lógica

Organizar o argumento de forma sequencial e coerente, apresentando cada passo de maneira clara e justificada, facilitando a verificação da validade.

Contextualização Adequada

Fornecer motivação e contexto quando apropriado, explicando a relevância do resultado e suas conexões com outros teoremas estabelecidos.

A Importância da Precisão

A falta de precisão não apenas compromete a formalização, mas pode invalidá-la completamente. Cada termo deve ser definido com rigor para evitar interpretações múltiplas.

Exemplo Ruim

"Seja S um conjunto e seja S ordenado."

Problema: Ambíguo — dado que S é um conjunto, o que significa "ordenado"? Conjuntos não possuem ordem inerente.

Exemplo Melhor

"Seja S um conjunto finito com uma relação de ordem $\leq$."

Solução: A especificidade elimina ambiguidades ao explicitar que existe uma relação de ordem definida.

❏ **Princípio fundamental:** Cada símbolo e conceito deve ser introduzido com definição clara antes de ser utilizado em demonstrações ou algoritmos.

Exemplo: Relação de Equivalência

Descrição Vaga e Imprecisa

"A relação de equivalência pode ser definida formalmente considerando um conjunto de pares ordenados que satisfazem três propriedades fundamentais: reflexividade, simetria e transitividade."

Problema: Usa linguagem informal que não especifica com precisão as propriedades matemáticas.

Definição Precisa

Definição: Uma relação de equivalência em um conjunto X é uma relação R que satisfaz:

- **Reflexividade:** $\forall x \in X, (x, x) \in R$
- **Simetria:** $\forall x, y \in X, (x, y) \in R \Rightarrow (y, x) \in R$
- **Transitividade:** $\forall x, y, z \in X, (x, y) \in R \text{ e } (y, z) \in R \Rightarrow (x, z) \in R$

Consistência e Boas Práticas

A escrita clara e bem estruturada de formalizações exige atenção a detalhes e consistência. Pequenas inconsistências podem invalidar demonstrações inteiras.

Evite Frases Longas

Use notação matemática sempre que possível para maior clareza e concisão.

Facilite a Leitura

Numere teoremas, lemas e resultados importantes para referência futura.

Destaque Conceitos-Chave

Use **negrito** ou *itálico* para termos técnicos e definições importantes.

Exemplo: Lista Invertida (Não-Formalizado)

"Uma lista invertida para um determinado termo é uma sequência de pares, onde o primeiro elemento em cada par é o identificador do documento e o segundo é a frequência de um termo em um documento referente ao identificador correspondente."

Problema: Descrição verbosa e ambígua.

Exemplo: Lista Invertida (Formalizado)

"Uma lista invertida para um termo t é uma sequência de pares na forma $\langle d, f \rangle$, onde d representa um documento e f é a frequência de t em d ."

Solução: Notação clara e concisa.

Exemplo de Inconsistência Corrigida

“

Com Problema de Consistência

"Seja $f(x)$ uma função definida em $\mathbb{R}$. Para todo $y \in \mathbb{R}$, existe x tal que $g(x) = y$."

Erro: A variável mudou de f para g sem aviso.

”

“

Problema Corrigido

"Seja $f(x)$ uma função definida em $\mathbb{R}$. Para todo $y \in \mathbb{R}$, existe x tal que $f(x) = y$."

Consistência mantida: Mesma notação ao longo da definição.

”

Provas Matemáticas: Conceito

Uma prova matemática é um argumento lógico rigoroso que estabelece a verdade de uma proposição de forma inquestionável. Ela constitui o coração da matemática, diferenciando-a de outras ciências através de seu padrão de certeza absoluta.

Existem diversos tipos de provas, cada uma adequada a diferentes situações:

- Prova direta: demonstra a conclusão seguindo logicamente das hipóteses
- Prova por contradição: assume a negação da conclusão e deriva uma contradição
- Prova por indução: estabelece casos base e passo indutivo
- Prova construtiva: fornece exemplo explícito do objeto cuja existência se afirma
- Prova por contrapositiva: demonstra que a negação da conclusão implica a negação da hipótese

Uma prova bem escrita deve ser completa, sem lacunas lógicas, mas também clara e compreensível. O equilíbrio entre rigor e legibilidade é essencial para comunicação efetiva.

📖 **Referências:** [1] S.G. Krantz, 2017, *A Primer of Mathematical Writing*. American Mathematical Society. [2] K. Houston, 2009, *How to Think Like a Mathematician: A Companion to Undergraduate Mathematics*. Cambridge University Press.

Estrutura de uma Prova

Provas matemáticas são o alicerce da validação formal. Uma prova rigorosa garante que uma proposição é verdadeira sob determinadas condições, eliminando dúvidas e estabelecendo certeza.

Uma demonstração bem estruturada melhora significativamente a compreensão e evita ambiguidades que podem surgir em argumentos informais.

Prova Direta

Parte das hipóteses e deduz a conclusão através de passos lógicos sequenciais

Prova por Casos

Divide o problema em casos exaustivos e prova cada um separadamente

Prova por Contradição

Assume o oposto da proposição e demonstra que isso leva a uma contradição

Prova por Indução

Prova para caso base e demonstra que se vale para n , também vale para $n+1$

"Uma prova é um argumento convincente que estabelece a veracidade de uma afirmação matemática."

— S.G. Krantz, *A Primer of Mathematical Writing*, 2017

Exemplo: Irrracionalidade de $\sqrt{2}$

A prova da irracionalidade de $\sqrt{2}$ é um exemplo elegante de demonstração por contradição, ilustrando como a formalização rigorosa estabelece verdades matemáticas.

Hipótese por Contradição

Suponha que $\sqrt{2} = \frac{p}{q}$, onde p e q são inteiros primos entre si (fração irredutível).

Dedução Adicional

Se p é par, então $p = 2k$ para algum inteiro k .

Substituindo: $(2k)^2 = 4k^2 = 2q^2$, logo $2k^2 = q^2$.

Manipulação Algébrica

Elevando ao quadrado ambos os lados: $2q^2 = p^2$

Logo, p^2 é par, o que implica que p também é par.

Contradição

Portanto, q^2 também é par, o que significa que q é par.

Contradição: p e q não podem ser ambos pares se eram primos entre si! ❗

📖 **Referências:** K. Houston (2009), *How to Think Like a Mathematician*; J. Zobel (2015), *Writing for Computer Science*

Pseudocódigo: Conceito

O pseudocódigo é uma ferramenta fundamental na ciência da computação, permitindo expressar algoritmos de forma independente de linguagens de programação específicas. Ele combina estruturas algorítmicas com linguagem natural, proporcionando clareza sem as restrições sintáticas de linguagens formais.

Características essenciais:

- Clareza e legibilidade superiores ao código real
- Independência de sintaxe específica de linguagens
- Foco na lógica e estrutura do algoritmo
- Organização hierárquica com indentação
- Ausência de ambiguidades na descrição

The WeightedEdit function computes the edit distance between two strings, assigning a higher penalty for errors closer to the front.

Input: S1, S2: strings to be compared.

Output: weighted edit distance

Variables: L1, L2: string lengths

F[L1, L2]: array of minimum distances

W: current weighting

M: maximum penalty

C: current penalty

WeightedEdit(S1, S2):

1. L1 = len(S1)
2. L2 = len(S2)
3. M = 2 × (L1 + L2)
4. F[0, 0] = 0
5. for i from 1 to L1
6. F[i, 0] = F[i - 1, 0] + M - i
7. for j from 1 to L2
8. F[0, j] = F[0, j - 1] + M - j
9. for i from 1 to L1
10. C = M - i
11. for j from 1 to L2
12. C = C - 1
13. F[i, j] = min(F[i - 1, j] + C,
 F[i, j - 1] + C,
 F[i - 1, j - 1] + C × isdiff(S1[i], S2[j]))
14. WeightedEdit = F[L1, L2]

O pseudocódigo deve ser suficientemente detalhado para permitir implementação direta em qualquer linguagem de programação, mas abstrato o suficiente para não se prender a idiosincrasias específicas. Convenções de indentação e nomenclatura clara de variáveis são cruciais para manter legibilidade.

Modularização em Pseudocódigo

A divisão de algoritmos complexos em funções modulares e reutilizáveis representa uma prática fundamental de engenharia de software e escrita algorítmica. O encapsulamento melhora significativamente a qualidade e manutenibilidade do código.

Modularidade

Cada função executa uma tarefa específica e bem definida, facilitando compreensão e teste isolado de componentes individuais.

Reutilização

Funções podem ser chamadas múltiplas vezes em diferentes contextos, evitando duplicação de código e reduzindo erros.

Abstração

Detalhes de implementação são ocultados, permitindo que usuários da função se concentrem apenas em sua interface e propósito.

Manutenibilidade

Alterações em funcionalidades específicas podem ser realizadas localmente sem afetar o restante do sistema.

```
SCHEDULER(Workflow W, ExecutionProcessesSet execProc)
1. FragmentSet fragSet = optimizeWorkflow(W)
2. while hasElements(fragSet)
3.   frag = getReadyFragment(fragSet)
4.   FAISet faiSet = generateActivations(frag)
5.   DispatchStrategy dispStrat = getDispatchStrategy(frag)
6.   for each FAI f in faiSet
7.     dispatch(f, dispStrat, execProc)
8.   fragSet = removeCompleted(fragSet, frag)
```

Ao projetar funções, é essencial escolher nomes descritivos, definir claramente parâmetros de entrada e saída, e documentar o comportamento esperado. Funções bem projetadas possuem responsabilidade única e interface simples.

📄 **Referência:** [1] E. Ogasawara, D. De Oliveira, P. Valduriez, J. Dias, F. Porto, e M. Mattoso, "An Algebraic Approach for Data-Centric Scientific Workflows", *Proceedings of the VLDB Endowment*, vol. 4, no 12, p. 1328–1339, 2011.

Exemplo: Algoritmo de Ordenação

A diferença entre descrições vagas e formais ilustra perfeitamente o poder da formalização matemática. Observe como a precisão transforma a compreensão do algoritmo:

Descrição Vaga

"O algoritmo percorre a lista e reorganiza os elementos até que fiquem ordenados."

Problema: Falta especificidade sobre como a reorganização ocorre e quando o processo termina.

Descrição Formal

"O algoritmo recebe uma sequência $S = [s_1, s_2, \dots, s_n]$ e aplica comparações entre elementos s_i e s_{i+1} , trocando-os se $s_i > s_{i+1}$

.

Esse processo continua até que S esteja ordenada segundo uma relação $\leq$. A complexidade do algoritmo, no pior caso, é $O(n^2)$."

Vantagem: Especifica claramente operações, condições e complexidade.

Prosecode

O Prosecode representa uma abordagem híbrida entre pseudocódigo formal e explicação em linguagem natural. Esta técnica foca na compreensão intuitiva do algoritmo, tornando-o acessível a uma audiência mais ampla enquanto mantém estrutura clara.

Características Distintivas

- Mistura linguagem natural com estrutura algorítmica
- Prioriza clareza conceitual sobre precisão sintática
- Facilita compreensão inicial antes de implementação formal
- Ideal para documentação e comunicação de ideias
- Reduz barreira de entrada para não-programadores

O Prosecode é particularmente útil em contextos educacionais e documentação técnica, onde o objetivo principal é transmitir a lógica e estratégia do algoritmo, não necessariamente sua implementação precisa.

WeightedEdit(s, t) compares two strings s and t , of lengths k_s and k_t , respectively, to determine their edit distance — the minimum cost in insertions, deletions, and replacements required to convert one into the other. These costs are weighted so that errors near the start of the strings attract a higher penalty than errors near the end.

We denote the i th character of string s by s_i . The principal internal data structure is a two-dimensional array F in which the dimensions have ranges from 0 to k_s , and from 0 to k_t , respectively. When the array is filled, $F_{i,j}$ is the minimum edit distance between the strings $s_1 \dots s_i$ and $t_1 \dots t_j$; and F_{k_s, k_t} is the minimum edit distance between s and t .

The value p is the maximum penalty, and the penalty for a discrepancy between positions i and j of s and t , respectively, is $p - i - j$, so that the minimum penalty is $p - k_s - k_t = p/2$ and the next-smallest penalty is $p/2 + 1$. Two errors, wherever they occur, will outweigh one.

- **(Set penalty.)** Set $p \leftarrow 2 \times (k_s + k_t)$
- **(Initialize data structure.)** The boundaries of array F are initialized with the penalty for deletions at start of string; for example, $F_{i,0}$ is the penalty for deleting i characters from the start of s . (a) Set $F_{0,0} \leftarrow 0$. (b) For each position i in s , set $F_{i,0} \leftarrow F_{i-1,0} + p - i$. (c) For each position j in t , set $F_{0,j} \leftarrow F_{0,j-1} + p - j$.
- **(Compute edit distance.)** For each position i in s and position j in t : (a) The penalty is $C = p - i - j$. (b) The cost of inserting a character into t (equivalently, deleting from s) is $I = F_{i-1,j} + C$. (c) The cost of deleting a character from t is $D = F_{i,j-1} + C$. (d) If $s_i = t_j$, the replacement cost is $R = F_{i-1,j-1}$. Otherwise, the replacement cost is $R = F_{i-1,j-1} + 1$. (e) Set $F_{i,j} \leftarrow \min(I, D, R)$.
- **(Return.)** Return F_{k_s, k_t} .

Formatação e Apresentação

A formatação adequada transforma conceitos complexos em conteúdo acessível e profissional.

Hierarquia Visual Clara

Utilize títulos, subtítulos e numeração para estabelecer a estrutura hierárquica do documento. Destaque definições, teoremas e provas usando formatação distinta, como negrito ou itálico para termos técnicos importantes.

Espaçamento Adequado

Mantenha espaçamento adequado entre parágrafos, equações e seções. Equações importantes devem ser centralizadas e numeradas para referência futura. O espaço em branco melhora legibilidade significativamente.

Referências e Citações

Inclua referências bibliográficas completas e consistentes. Cite adequadamente resultados conhecidos e trabalhos anteriores, demonstrando conhecimento do contexto histórico e acadêmico do tema.

Exemplos Ilustrativos

Forneça exemplos concretos após definições abstratas e teoremas gerais. Exemplos ajudam a consolidar compreensão e demonstram aplicabilidade prática dos conceitos apresentados.

Um documento bem formatado não apenas facilita a leitura, mas também demonstra profissionalismo e respeito pelo leitor. A consistência na formatação ao longo de todo o texto é essencial para manter credibilidade acadêmica.

IA como Parceira da Sua Pesquisa

Prompts para usar enquanto você desenvolve seu trabalho

Substitua os [colchetes] pelo contexto do seu próprio trabalho antes de usar qualquer prompt abaixo.

1. Classificar a Contribuição

Use quando... você precisa identificar se seu trabalho é teórico, empírico, metodológico ou aplicado.

"Meu trabalho trata de [tema/problema]. Ele propõe [solução/abordagem]. Com base nos critérios de formalização matemática e computacional — como definições rigorosas, proposições e provas — que tipo de contribuição científica meu trabalho representa? Justifique com base na estrutura formal do argumento."

2. Posicionar na Literatura

Use quando... você quer entender como seu trabalho se relaciona com o que já foi publicado.

"Estou desenvolvendo um trabalho sobre [tema]. Os principais conceitos formais que utilizo são [conceitos, ex: relações de equivalência, pseudocódigo, provas por contradição]. Quais lacunas na literatura meu trabalho pode preencher? Como devo posicionar minha contribuição em relação a trabalhos existentes sobre [área]?"

3. Escolher a Metodologia

Use quando... você está decidindo como estruturar sua abordagem de pesquisa.

"Meu objetivo é [objetivo do trabalho]. Preciso formalizar [conceito/algoritmo/sistema] usando linguagem matemática rigorosa. Que metodologia de pesquisa — dedutiva, indutiva, experimental — é mais adequada? Como devo estruturar a seção de metodologia para que ela seja consistente com os princípios de escrita formal?"

4. Antecipar Críticas da Banca

Use quando... você quer se preparar para a defesa ou revisão do trabalho.

"Minha proposição central é: [enuncie a proposição]. A prova/argumento que apresento segue a estrutura [descreva brevemente]. Quais são as principais críticas que uma banca poderia levantar sobre a precisão, consistência ou completude da minha formalização? Como posso fortalecer meu argumento?"

5. Revisar a Escrita Científica

Use quando... você terminou um trecho e quer melhorar a clareza e o rigor formal.

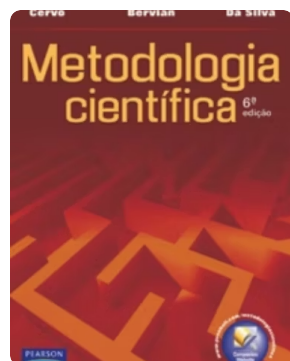
"O trecho abaixo faz parte da minha [seção: introdução/metodologia/conclusão]. Ele trata de [tema]. Avalie se a linguagem é precisa, se os termos técnicos estão bem definidos, se há ambiguidades e se a estrutura lógica está clara. Sugira melhorias mantendo o estilo de escrita formal acadêmica: [cole o trecho aqui]."

  O LLM não conhece sua pesquisa — você precisa fornecer o contexto. Sempre confronte as respostas com a literatura real e com seu orientador. Referências geradas por IA devem ser verificadas antes de qualquer uso.

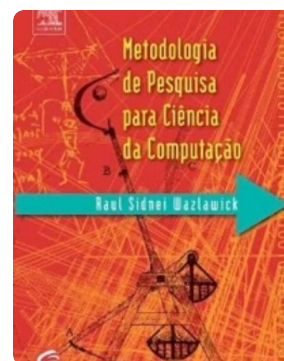
Referências Bibliográficas



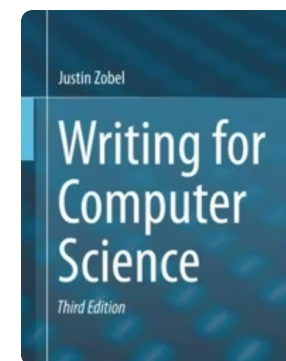
Perovano (2016)
Manual de metodologia da pesquisa científica - Editora Intersaberes.



Cervo, Bervian & Silva (2006)
Metodologia Científica - Pearson Universidades.



Wazlawick (2017)
Metodologia de Pesquisa para Ciência da Computação - Elsevier Brasil.



Zobel (2015)
Writing for Computer Science - Springer.